
Chapter 14: Transactions



Chapter 14: Transactions

- ▶ A **collection of several operations** on the database appears to be *a single unit for the database user*, for example, a transfer of funds from checking account to a savings account is a single operation from the customer's standpoint.
- ▶ All these operations occur, or none occur. These operations are called *transactions*.
- ▶ In this chapter, we will introduce the basic concept of *transaction processing*.



Chapter 14: Transactions

- ▶ **Transaction Concept**
- ▶ **Storage Structure**
- ▶ **Transaction State**
- ▶ **Serializability**
- ▶ **Transaction Isolation Levels**
- ▶ **Implementation of Isolation**



Transaction Concept

- ▶ A **transaction** is a *unit of program execution* that accesses and possibly updates various data items.
- ▶ A transaction is initiated by a user program written in a *high-level data-manipulation language* (typically SQL), or *programming language* (C++, or Java), with embedded database accesses in JDBC or ODBC.
- ▶ A transaction is delimited by statements (or function calls) of the form *begin transaction* and *end transaction*.
- ▶ The transaction consists of all operations executed between the begin transaction and end transaction.



Transaction Concept (Cont.)

- ▶ We require that the database system maintain the following *properties* of the transactions
 - ▶ **Atomicity**. Either **all** operations of the transaction are *reflected* properly in the database or **none** are.
 - ▶ **Consistency**. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
 - ▶ If a transaction is run atomically in isolation starting from a *consistent database*, the database must again be *consistent* at the end of the transaction.



Transaction Concept (Cont.)

- ▶ We require that the database system maintain the following *properties* of the transactions
 - ▶ **Isolation.** Although multiple transactions may execute *concurrently*, but the system guarantees that,
 - ▶ for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished. Thus, each transaction is *unaware of other transactions executing concurrently in the system*.
 - ▶ **Durability.** After a transaction completes successfully, the changes it has made to the database *persist*, even if there are system failures.
- ▶ These properties are often called the **ACID properties**

A Simple Transaction Model

- ▶ Here, we shall illustrate the transaction concept using a simple *bank application* consisting of *several accounts* and *a set of transactions* that access and update those accounts.
- ▶ *Transaction access data* using *two operations*:
 - ▶ *read(X)*: which transfers the data item *X* *from the database to a variable*, also called *X*, in *a buffer in main memory* belonging to the transaction that executed the *read* operation
 - ▶ *write(X)*: which transfers the value in the variable *X* in the *main-memory buffer* of the transaction that executed the *write* to the data item *X* in the *database*.

A Simple Transaction Model (Cont.)

► **Note**

- If a change to a data item *appears* only in *main memory*, or if it has been written to the database *on disk*.
- In a real database system
 - The write operation *does not necessarily* result in the immediate *update* of the data *on the disk*;
 - The write operation may be *temporarily stored elsewhere* and *executed on the disk later*.
- For now, however, we shall *assume* that the *write operation updates the database immediately*.



A Simple Transaction Model (Cont.)

- ▶ T_i be a transaction that to *transfer \$50 from account A to account B*

T_i : **read**(A);
 $A := A - 50$;
 write(A);
 read(B);
 $B := B + 50$;
 write(B).

- ▶ *Consistency requirement*
 - ▶ The consistency requirement here is that the *sum of A and B is unchanged* by the execution of the transaction.
 - ▶ Without the consistency requirement, money could be *created* or *destroyed* by the transaction!



A Simple Transaction Model (Cont.)

Atomicity requirement

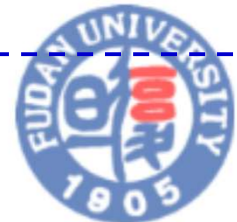
During the execution of transaction T_i , a failure occurs:

- suppose that the failure happened *after the write(A)*, but *before the write(B)*.
- In this case, the values of accounts A and B reflected in the database are \$950 and \$2000.
- The system destroyed \$50 as a result of this failure. The sum $A + B$ is *no longer preserved*.
- The failure leads to an *inconsistent state*.

```
 $T_i$  : read(A);  
      A := A - 50;  
      write(A);  
      read(B);  
      B := B + 50;  
      write(B).
```

fail

Suppose: account A : \$1000
account B : \$2000



A Simple Transaction Model (Cont.)

▶ Atomicity requirement (Cont.)

- ▶ We must ensure that such *inconsistencies are not visible* in a database system.
- ▶ However, that the system must *at some point* be in an inconsistent state.
 - ▶ For example: if transaction T_i is executed to completion, there exists a point at which the values of accounts A and B are \$950 and \$2000. It is clearly an inconsistent state.
- ▶ That is the reason for the *atomicity requirement*: if the *atomicity property* is present, *all* actions of the transaction are reflected in the database, or *none* are.



A Simple Transaction Model (Cont.)

- ▶ *Basic idea behind ensuring atomicity*
 - ▶ The DB system *keeps track (on disk) of the old values* of any data on which a transaction performs a write.
 - ▶ This information is written to a file called the *log*.
 - ▶ If transaction does not complete its execution, the DB system *restores the old values* from the log to make it appear as though the transaction never executed.



A Simple Transaction Model (Cont.)

- ▶ **Durability requirement**

- ▶ Once the user has been notified that the transaction has completed successfully (i.e., transfer of the \$50 has taken place), it must be the case that *no system failure can result in a loss* of data corresponding to this.

- ▶ *To guarantee durability*

- ▶ The updates carried out by the transaction have been *written to disk* before the transaction completes.
- ▶ Information about the updates carried out by the transaction and written to disk is *sufficient to enable the database to reconstruct the updates* when the database system is restarted after the failure.



A Simple Transaction Model (Cont.)

- ▶ **Isolation requirement**

- ▶ Even if the *consistency* and *atomicity* properties are ensured for each transaction,
- ▶ But, if several transactions are *executed concurrently*, their operations may *interleave in some undesirable way, resulting in an inconsistent state.*

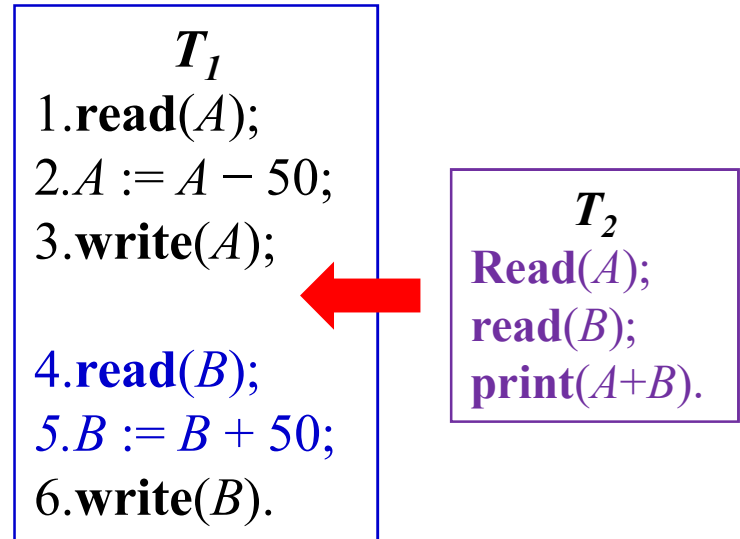


A Simple Transaction Model (Cont.)

► Isolation requirement

► Example: T_1 and T_2 :

☞ Between steps 3 and 6, with the deducted total written to A and the increased total **yet** to be written to B .



- Another transaction T_2 reads A and B at this *intermediate point* and computes $A+B$, it will observe an inconsistent value.
- Furthermore, if T_2 then performs *updates on A and B based on the inconsistent values* that it read, the database may be left in an inconsistent even after both transactions have completed.



A Simple Transaction Model (Cont.)

► Isolation requirement

- A way to avoid the problem of *concurrently executing transactions* is to execute transactions serially: *one after the other*.
- However, concurrent execution of transactions *provides significant performance benefits*. So other solutions have therefore been developed; they allow multiple transactions to execute concurrently.



Storage Structure

- ▶ To understand how to *ensure the atomicity and durability*, we must understand *how the data is stored and accessed*. We first review some storage terms
 - ▶ *Volatile storage*: information does *not survive system crashes* (E.g., main memory, cache memory).
 - ▶ *Nonvolatile storage*: information *survives system crashes* (E.g., magnetic disk, flash storage). But *hardware failure* may result in loss of information.



Storage Structure (Cont.)

- ▶ To understand how to *ensure the atomicity and durability*, we must understand *how the data is stored and accessed*. We first review some storage terms
 - ▶ *Stable storage*: information is *never lost*.
 - ▶ Stable storage is theoretically impossible to obtain, it can be closely approximated by techniques that make data loss extremely unlikely.
 - ▶ To implement stable storage, We *replicate* the information in several *nonvolatile storage media* (usually disk) with *independent failure modes*. Updates must be done with care to ensure that no information is lost.



Storage Structure (Cont.)

- ▶ For a transaction to be *durable*, its changes need to be written to *stable storage*.
- ▶ Similarly, for a transaction to be *atomic*, *log records need to be written to stable storage before any changes are made to the database on disk.*



Transaction State

- ▶ We need to be more precise about what we mean by *successful completion* of a transaction.
- ▶ For instance, we can establish a simple *abstract transaction model*, that is, a transaction must be in one of the following states
 - ▶ *Active* – the initial state; the transaction stays in this state while it is executing
 - ▶ *Partially committed* – after the final statement has been executed.
 - ▶ *Failed* – after the discovery that normal execution can no longer proceed.



Transaction State (Cont.)

- ▶ *Aborted* – after the transaction has been *rolled back* and the database restored to its state prior to the start of the transaction.
- ▶ To ensure the *atomicity property*, an aborted transaction must have no effect on the state of the database.
- ▶ Once the changes caused by an aborted transaction have been undone, we say that the transaction has been *rolled back*.
- ▶ It is part of the responsibility of the recovery scheme to manage transaction aborts.
- ▶ This is done typically by maintaining a *log*.



Transaction State (Cont.)

- ▶ *Aborted* – after the transaction has been *rolled back* and the database restored to its state prior to the start of the transaction.
- ▶ A transaction in *aborted state*, at this point, its two options
 - ▶ *restart the transaction*
 - *some hardware* or *software error*
 - A restarted transaction is regarded as a *new transaction*
 - ▶ *kill the transaction*
 - *some internal logical error* that can be corrected only by rewriting the application program.
 - the *input was bad*, or because the *desired data were not found in the database*.



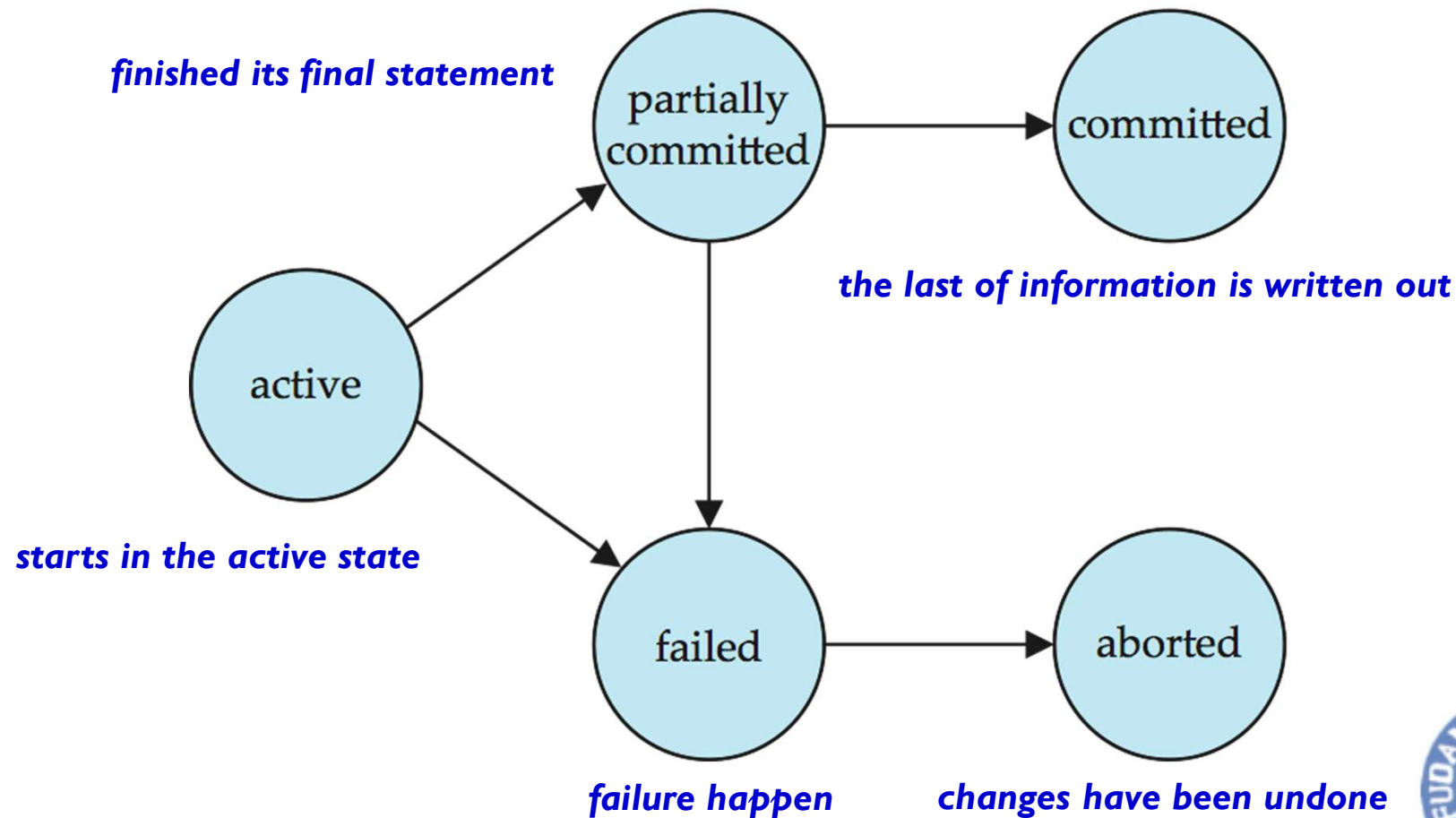
Transaction State (Cont.)

- ▶ *Committed* – *after successful completion*
 - ▶ A committed transaction that has performed updates transforms the database *into a new consistent state*, which *must persist* even if there is a system failure.
 - ▶ Once a transaction has committed, we *cannot undo its effects* by aborting it.
 - ▶ The *only way to undo the effects of a committed* transaction is to execute a *compensating transaction*
- ▶ **[Note]** A transaction is said to have *terminated* if it has either *committed* or *aborted*.

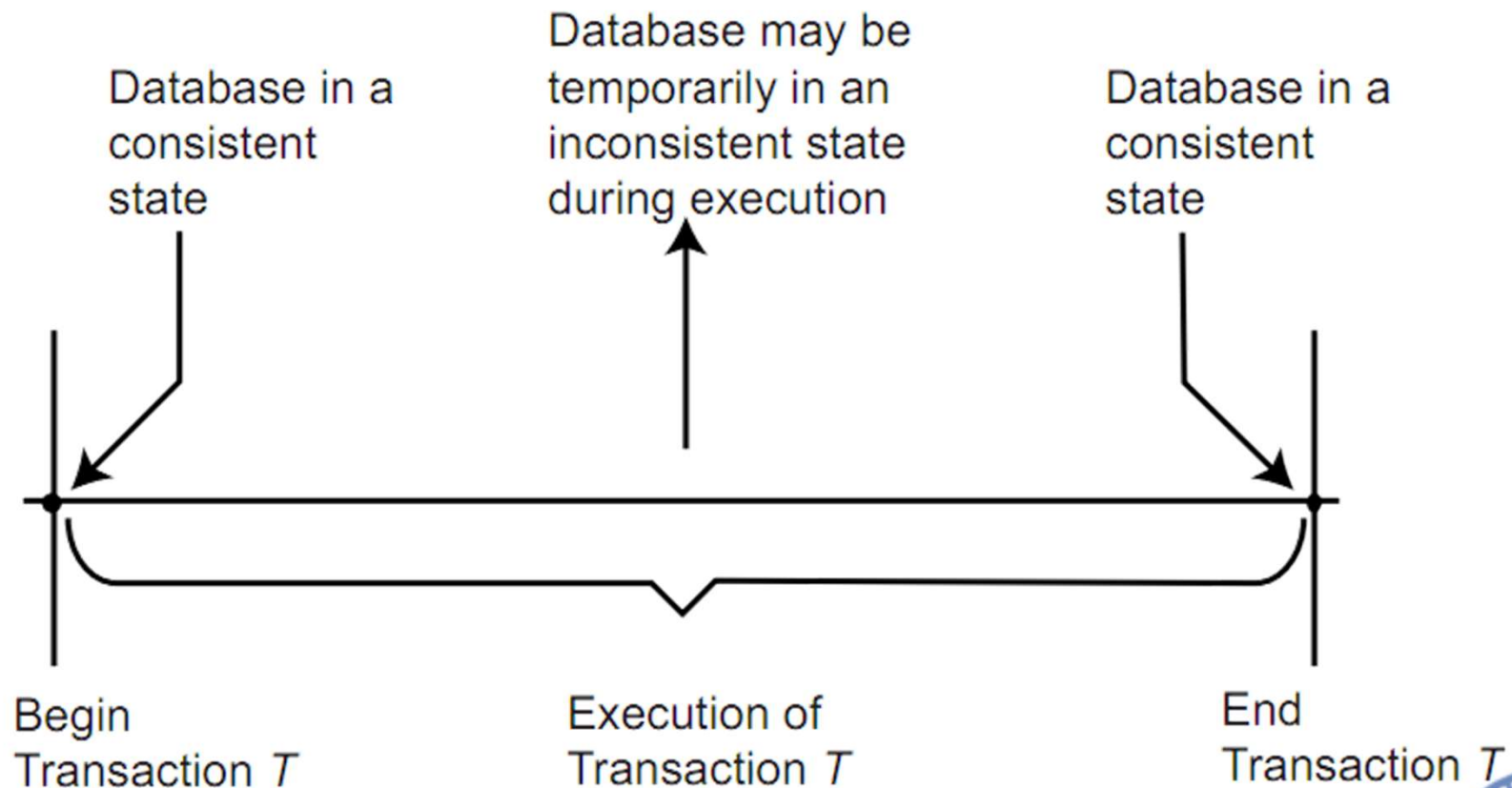


Transaction State (Cont.)

State diagram of a transaction



A Transaction Model



Transaction State (Cont.)

- ▶ We must be cautious when dealing with *observable external writes*, such as writes to a user's screen. Once such a write has occurred, it *cannot be erased*.
- ▶ Most systems allow such writes to take place only after the transaction has entered the *committed state*.
 - ▶ One way to implement such a scheme is for the database system to store any value associated with *such external writes temporarily in a special relation in the database*, and to perform the actual writes only after the transaction enters the committed state.
 - ▶ If the system **fail** after the transaction has *entered the committed state*, but *before completing the external writes*, the database system will carry out the external writes (*using the data in nonvolatile storage*) when the system is restarted.

Transaction Isolation

- ▶ *Multiple transactions are allowed to run concurrently* in the system. Certainly, it is much easier to make transactions run serially. But *advantages* for allowing concurrency are:
 - ▶ *Improved throughput and resource utilization*
 - ▶ I/O activity can be done in parallel with processing at the CPU.
 - ▶ The parallelism of the CPU and the I/O system can therefore be exploited to run multiple transactions in parallel.
 - ▶ Increasing the number of transactions executed in a given time – *throughput*.
 - ▶ The processor and disk spend less time idle – increasing the *utilization*.

Transaction Isolation (Cont.)

- ▶ *Multiple transactions are allowed to run concurrently* in the system. Certainly, it is much easier to make transactions run serially. But *advantages* for allowing concurrency are:
 - ▶ **Reduced waiting time**
 - ▶ There may be a mix of transactions running on a system, some short and some long.
 - ▶ Serial execution leads to *unpredictable delays*. Because a short transaction may have to wait for a preceding long transaction to complete.



Transaction Isolation (Cont.)

- ▶ *Multiple transactions are allowed to run concurrently* in the system. Certainly, it is much easier to make transactions run serially. But *advantages* for allowing concurrency are:
 - ▶ **Reduced waiting time**
 - ▶ If the transactions are operating on different parts of the database, concurrent execution can reduce *unpredictable delays*.
 - ▶ Also reducing the *average response time* -- the average time for a transaction to be completed after it has been submitted



Transaction Isolation (Cont.)

- ▶ *Concurrency control schemes*

- ▶ Mechanisms to achieve isolation, that is, to *control the interaction among the concurrent transactions* in order to *prevent them from destroying the consistency of the database*



Schedules

- ▶ *Schedule* – a sequences of instructions that specify the *chronological order* in which instructions of concurrent transactions are executed in the system.
- ▶ A schedule for a set of transactions must consist of *all instructions* of those transactions. It must *preserve the order* in which the instructions appear in *each individual transaction*.
- ▶ A transaction that *successfully completes* its execution will have a *commit instruction* as the last statement (by default transaction assumed to execute the commit instruction as its last step).
- ▶ A transaction that *fails to successfully complete* its execution will have an *abort instruction* as the last statement.



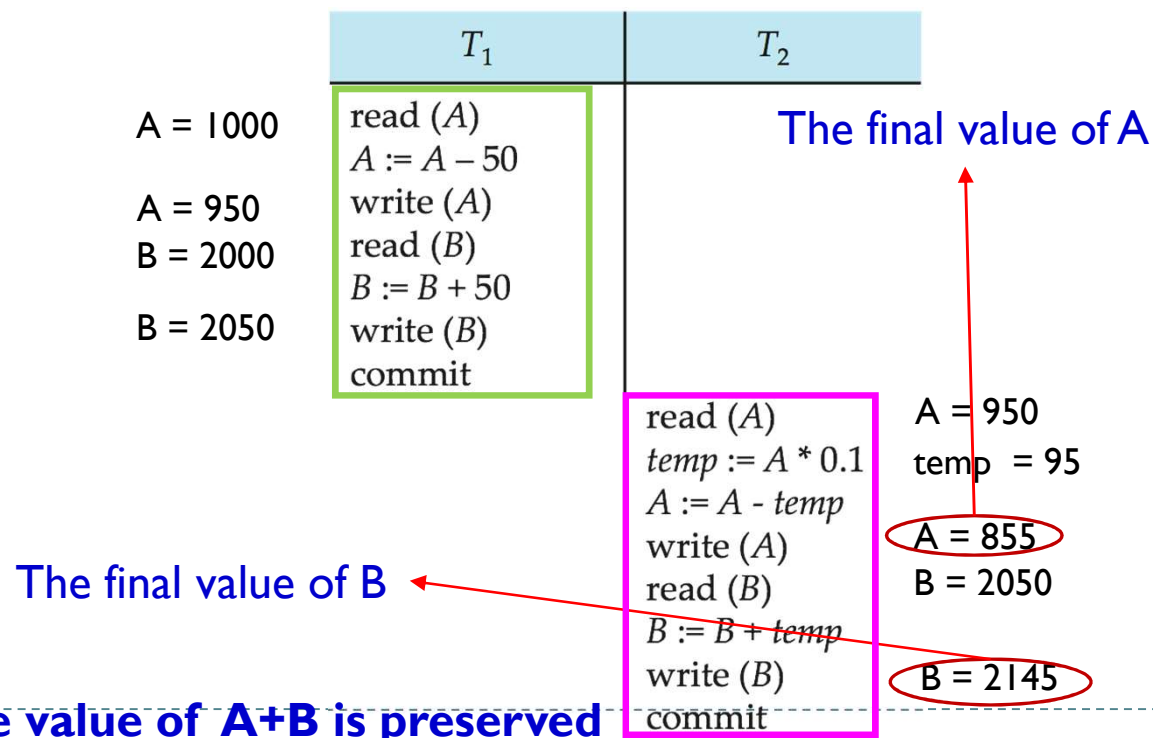
Schedules (Cont.)

- ▶ A transaction that *successfully completes* its execution will have a *commit instruction* as the last statement (by default transaction assumed to execute commit the instruction as its last step).
- ▶ A transaction that *fails to successfully complete* its execution will have an *abort instruction* as the last statement.



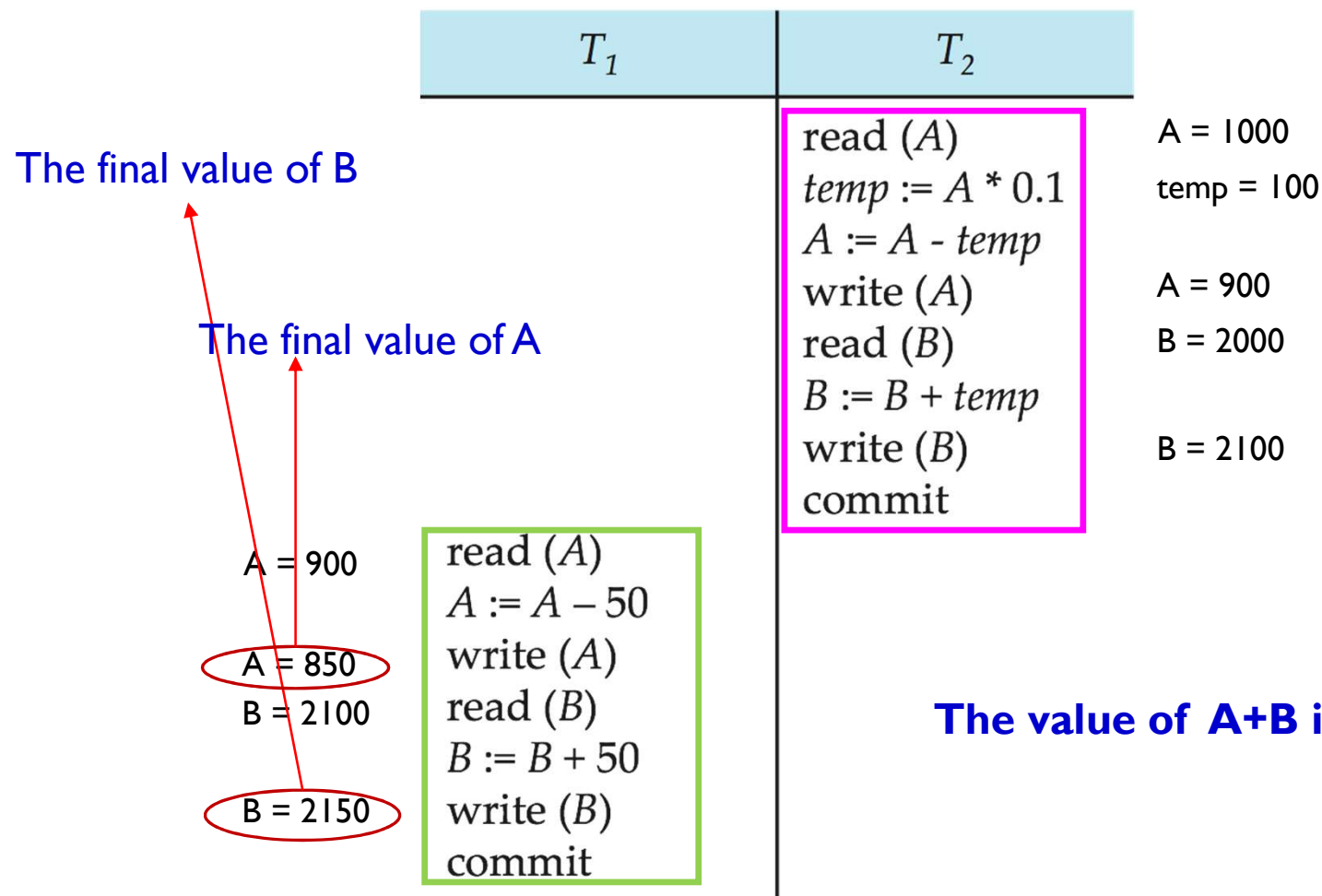
Schedule 1

- ▶ Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B . The initial values of accounts A and B are \$1000 and \$2000.
- ▶ A **serial schedule** in which T_1 is followed by T_2 ($T_1 \rightarrow T_2$):



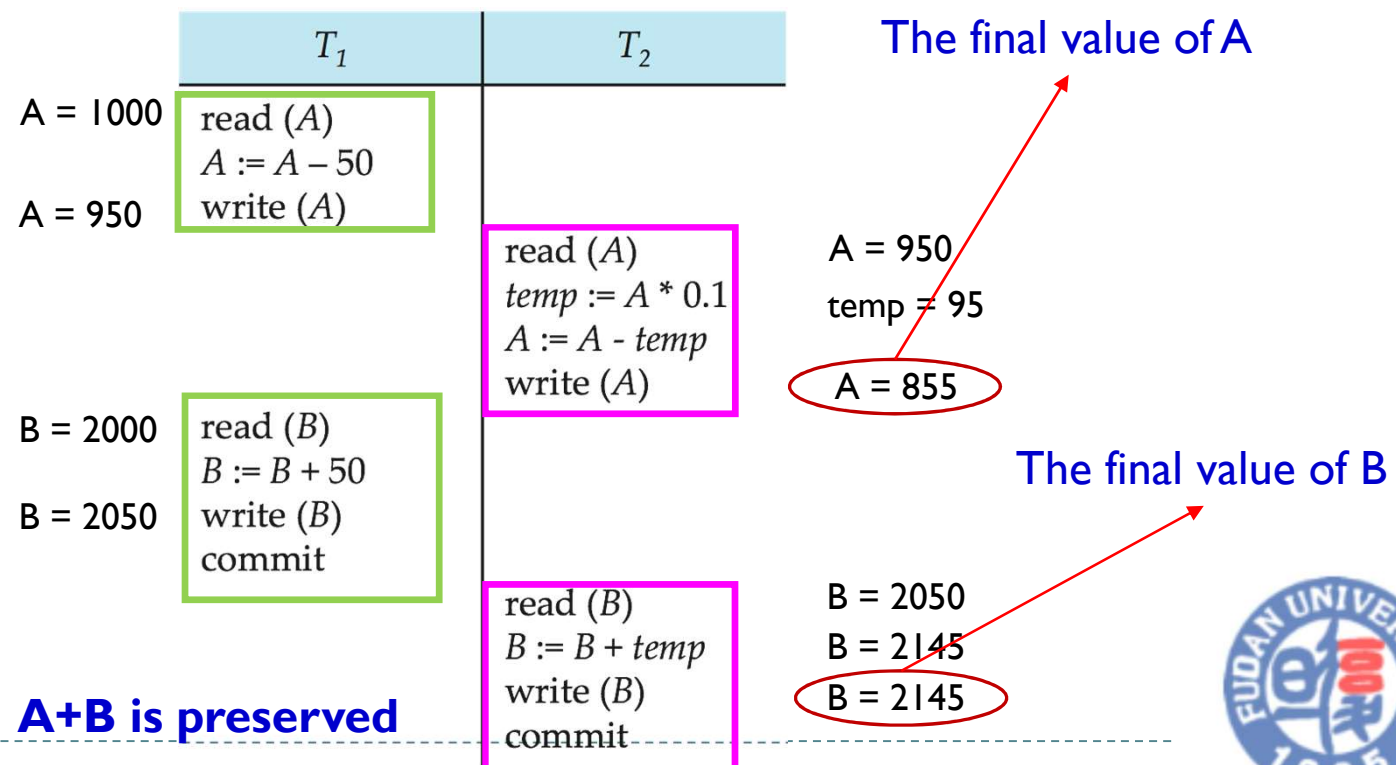
Schedule 2

- A **serial schedule** where T_2 is followed by T_1 ($T_2 \rightarrow T_1$):



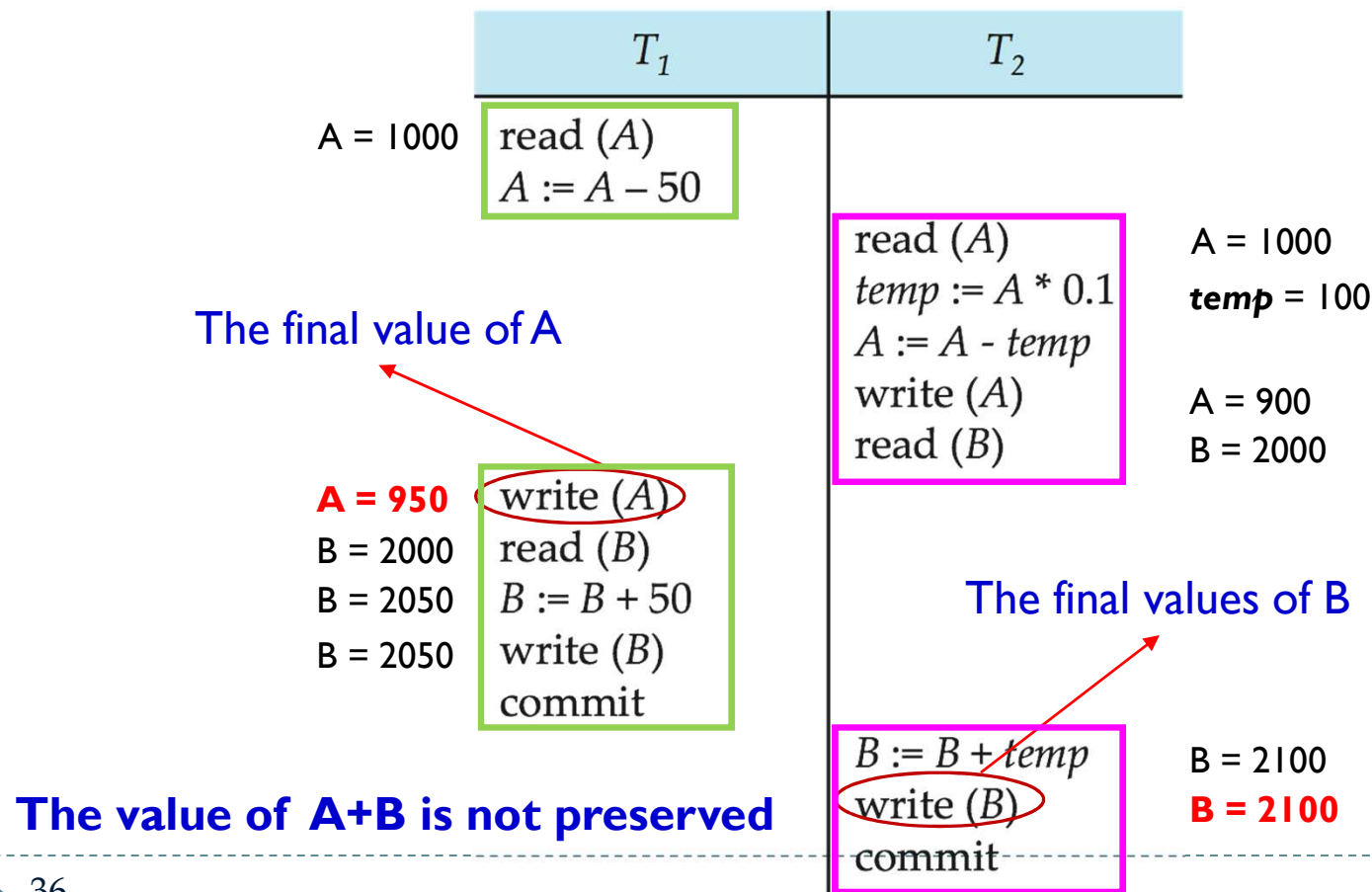
Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is *not a serial schedule*, but it is *equivalent* to Schedule 1.



Schedule 4

- ▶ The following concurrent schedule *does NOT preserve the value of $(A + B)$* .



Serializability

- ▶ The schedule which is *equivalent to a serial schedule* is called a *serializable schedule*.
- ▶ We ignore the various types of operations, but instead consider only two operations: *read* and *write*
 - ▶ We assume that, between a *read(Q)* instruction and a *write(Q)*, a transaction may *perform arbitrary sequence of operations* on the copy of *Q* that is residing in the local buffer of the transaction.
 - ▶ In this mode, the *only significant operations*, from a scheduling point of view, are *read* and *write* instructions.
- ▶ We will discuss different forms of *schedule equivalence*, but focus on a particular form called *conflict serializability*.

Conflicting Instructions

- ▶ Consider a schedule S :
 - ▶ I and J are two **consecutive instructions** of transactions T_i and T_j , respectively ($i \neq j$).
 - ▶ If I and J refer to *different data items*, then we can swap I and J *without affecting the results* of any instruction in the schedule.
 - ▶ But, if I and J refer to the *same data item* Q , **four cases** are needed to be considered:
 - $I = \text{read}(Q), J = \text{read}(Q)$
 - $I = \text{read}(Q), J = \text{write}(Q)$
 - $I = \text{write}(Q), J = \text{read}(Q)$
 - $I = \text{write}(Q), J = \text{write}(Q)$



Conflicting Instructions (Cont.)

1. $I = \text{read}(Q), J = \text{read}(Q)$. The order of I and J does not matter. The same value of Q is read.
2. $I = \text{read}(Q), J = \text{write}(Q)$. The order of I and J matters.
 - a. If I comes before J , then T_i does not read the value of Q that is written by T_j in instruction J .
 - b. If J comes before I , then T_i reads the value of Q that is written by T_j .
3. $I = \text{write}(Q), J = \text{read}(Q)$. The order of I and J matters for reasons similar to 2.



Conflicting Instructions (Cont.)

4. $I = \text{write}(Q)$, $J = \text{write}(Q)$. The order of I and J matters.
 - a. The result of only *the latter of the two write instructions* is preserved in the database, so the value obtained by the next $\text{read}(Q)$ instruction of S is affected.
 - b. If there is no other $\text{write}(Q)$ instruction after I and J in S , then *the order of I and J directly affects the final value of Q* in the database state that results from schedule S .
- I and J **conflict** if they are operations by *different transactions* on the *same data item*, and at least one of these instructions is a *write* operation.



Conflict Serializability

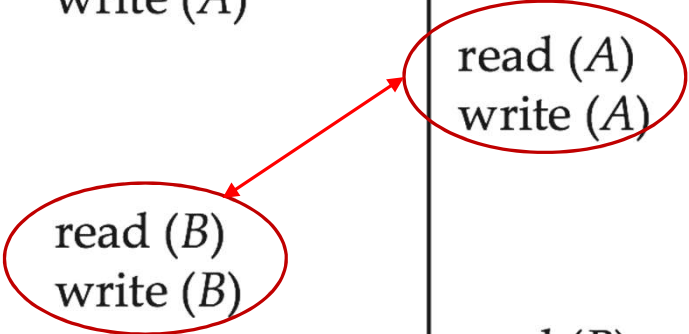
- ▶ If a *schedule S* can be transformed into a *schedule S'* by *a series of swaps of non-conflicting instructions*, we say that S and S' are **conflict equivalent**.
- ▶ We say that a schedule S is **conflict serializable** if it is *conflict equivalent to a serial schedule*.



Conflict Serializability (Cont.)

- ▶ Schedule 3 can be transformed into Schedule 6 by *series of swaps of non-conflicting instructions*.
- ▶ Schedule 6 is a serial schedule (T_2 follows T_1).
- ▶ Therefore Schedule 3 is **conflict serializable**.

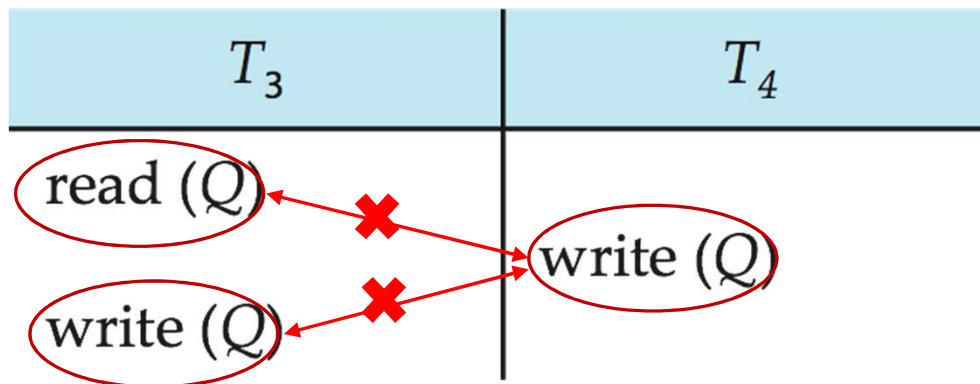
T_1	T_2	T_1	T_2
read (A) write (A)	read (A) write (A)	read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)
read (B) write (B)	read (B) write (B)		



schedule 3
schedule 6


Conflict Serializability (Cont.)

- ▶ Example of a schedule that is **NOT** conflict serializable



- ▶ We are **unable** to *swap instructions* in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.

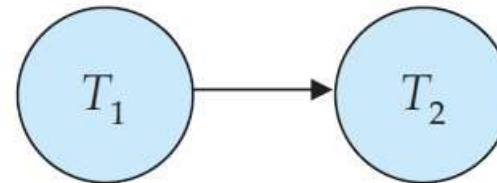
Test for Serializability

- ▶ Considering a schedule of a set of transactions $T_1, T_2, \dots, T_n$, we can construct a **precedence graph** to determine **conflict serializability** of a schedule.
- ▶ **Precedence graph** a directed graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges.
 - ▶ The set of **vertices** consists of all the **transactions**.
 - ▶ The set of **edges** consists of all edges $T_i \rightarrow T_j$ for which one of three conditions holds:
 1. T_i executes **write(Q)** before T_j executes **read(Q)**.
 2. T_i executes **read(Q)** before T_j executes **write(Q)**.
 3. T_i executes **write(Q)** before T_j executes **write(Q)**.

Examples of Precedence Graph

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

T_1 execute all conflict instructions first. For example, T_1 execute **write(B)** before T_2 execute **read(B)**.



Precedence graph for schedule 1

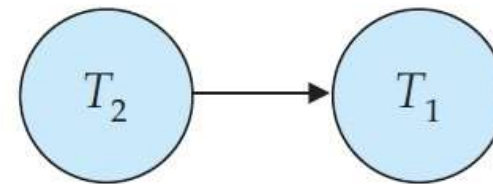
schedule 1

Examples of Precedence Graph (Cont.)

T_1	T_2
read (A) $A := A - 50$ write (A) <u>read (B)</u> $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ <u>write (B)</u> commit

schedule 2

T_2 execute all conflict instructions first. For example, T_2 execute **$write(B)$** before T_1 execute **$read(B)$** .

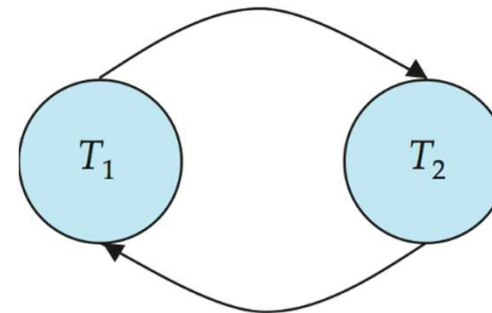


Precedence graph for schedule 2

Examples of Precedence Graph (Cont.)

T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	$B := B + temp$ write (B) commit

T_1 executes **read(A)** before T_2 executes **write(A)** and T_2 executes **write(A)** before T_1 executes **write(A)**.

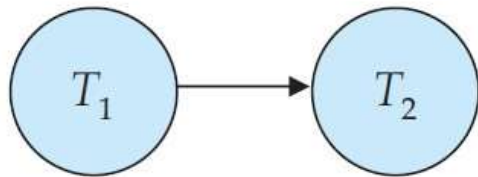


Precedence graph for schedule 4

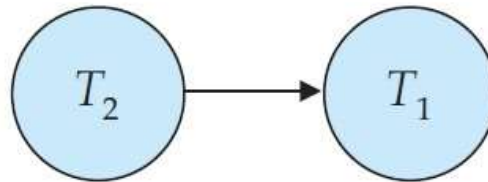
schedule 4

Test for Conflict Serializability

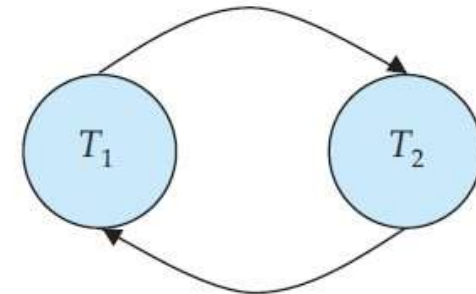
- ▶ If the *precedence graph* for schedule S has a *cycle*, then schedule S is *not conflict serializable*. If the graph contains no cycles, then the schedule S is *conflict serializable*.



schedule 1
conflict serializable



schedule 2
conflict serializable

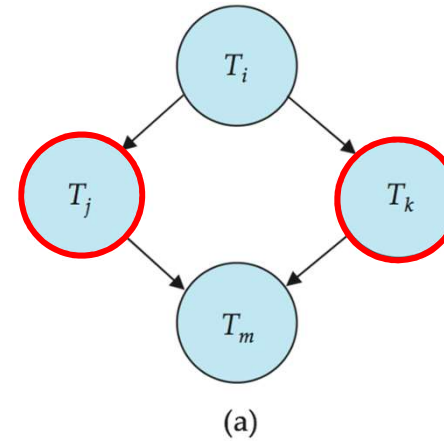


schedule 4
not conflict serializable

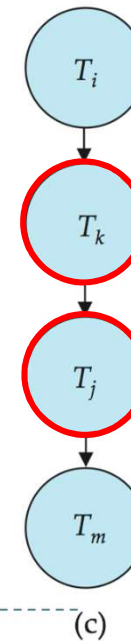
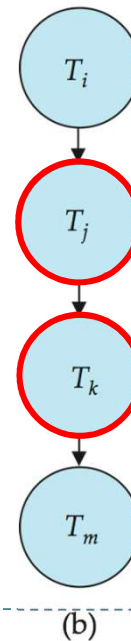
Test for Conflict Serializability (Cont.)

- ▶ A *serializability order* of the transactions can be obtained by *finding a linear order consistent with the partial order of the precedence graph*. This process is called **topological sorting**.
- ▶ There are, in general, *several possible linear orders* that can be obtained through a topological sort.

Test for Conflict Serializability (Cont.)



E.g., the graph of (a) has the two acceptable linear orderings shown in (b) and (c).



Test for Conflict Serializability (Cont.)

- ▶ To test for *conflict serializability*, we need to construct the *precedence graph* and to invoke a *cycle-detection algorithm*.
- ▶ **Cycle-detection algorithms**, such as those based on *depth-first search*, require on the order of n^2 operations, where n is the number of vertices in the graph.



Test for Conflict Serializability (Cont.)

- ▶ It is possible to have two schedules that produce the *same outcome*, but that are *not conflict equivalent*.
- ▶ For example, consider transaction T_5 , which transfers \$10 from account B to account A .

☞ It is easy to find that schedule 8 is *not conflict equivalent to the serial schedule $\langle T_1, T_5 \rangle$* .

schedule 8

T_1	T_5
$A = 1000$ read(A)	
$A := A - 50$	
$A = 950$ write(A)	
	read(B) $B = 2000$
	$B := B - 10$
	write(B) $B = 1990$
$B = 1990$ read(B)	
$B := B + 50$	
$B = 2040$ write(B)	
	read(A) $A = 950$
	$A := A + 10$
	write(A) $A = 960$

Test for Conflict Serializability (Cont.)

- It is possible to have two schedules that produce the *same outcome*, but that are *not conflict equivalent*.
- For example, consider transaction T_5 , which transfers \$10 from account B to account A .

- Since, **write**(B) of T_5 conflicts with the **read**(B) of T_1 creating edge $T_5 \rightarrow T_1$ in the precedence graph.
- write**(A) of T_1 conflicts with the **read**(A) of T_5 creating edge $T_1 \rightarrow T_5$ in the precedence graph.
- The precedence graph has a cycle and that schedule 8 is not serializable.

schedule 8

T_1	T_5
A = 1000 read(A) A := A - 50 A = 950 write(A)	read(B) B = 2000 B := B - 10 write(B) B = 1990
B = 1990 read(B) B := B + 50 B = 2040 write(B)	read(A) A = 950 A := A + 10 write(A) A = 960

Test for Conflict Serializability (Cont.)

- It is possible to have two schedules that produce the *same outcome*, but that are *not conflict equivalent*.
- For example, consider transaction T_5 , which transfers \$10 from account B to account A .

But the final values of accounts A and B after the execution of either schedule 8 or the serial schedule $\langle T_1, T_5 \rangle$ are the same—960 and 2040, respectively.

schedule 8

T_1	T_5
$A = 1000$ read(A) $A := A - 50$ $A = 950$ write(A)	$B = 2000$ read(B) $B := B - 10$ $B = 1990$ write(B)
$B = 1990$ read(B) $B := B + 50$ $B = 2040$ write(B)	$A = 950$ read(A) $A := A + 10$ $A = 960$ write(A)

Test for Conflict Serializability (Cont.)

- ▶ We can see from this example that there are *less-stringent definitions of schedule equivalence than conflict equivalence*.
- ▶ For the system to determine schedule 8 produces the same outcome as the serial schedule $\langle T_1, T_5 \rangle$, it must *analyze the computation* performed by T_1 and T_5 , *rather than just the read and write operations*. A lot of computation is needed.



Test for Conflict Serializability (Cont.)

- ▶ There are other definitions of *schedule equivalence* based *purely on the read and write operations*.
- ▶ One such definition is *view equivalence*, a definition that leads to the concept of *view serializability*.
- ▶ It is not used in practice due to its *high degree of computational complexity*.



Transaction Isolation and Atomicity

- ▶ So far, we have studied schedules while assuming implicitly that there are *NO transaction failures*. But failures always happen.
- ▶ If a transaction T_i fails, we need to *undo the effect of this transaction* to ensure the *atomicity property* of the transaction.
- ▶ In a system that allows concurrent execution, the *atomicity property* requires that any transaction T_j that is *dependent* on T_i (that is, T_j has read data written by T_i) is also aborted.



Recoverable Schedules

T_6	T_7
read(A) write(A)	
	read(A) commit
read(B)	

schedule 9

- ▶ The above schedule is called a *partial schedule* because we have not included a *commit* or *abort* operation for T_6 .
- ▶ T_7 has read the value of data item A written by T_6 . Therefore, we say that T_7 is *dependent* on T_6 .
- ▶ Suppose that T_6 fails before it commits, we must abort T_7 to ensure atomicity. However, T_7 has already committed and cannot be aborted.

Recoverable Schedules (Cont.)

- ▶ **Recoverable schedule** — if a transaction T_j *reads* a data item previously *written* by a transaction T_i , then the *commit* operation of T_i appears before the *commit* operation of T_j .

T_i write $\rightarrow T_j$ read $\Rightarrow T_i$ commit $\rightarrow T_j$ commit

- ▶ Schedule 9 is a **non-recoverable** schedule, because T_7 *read* A and is *committed* before T_6 .
- ▶ For the example of schedule 9 to be recoverable, T_7 **would have to delay committing until after T_6 commits.**

T_6	T_7
read(A)	
write(A)	
	read(A)
	commit
read(B)	

schedule 9

Cascading Rollbacks

- ▶ **Cascading rollback** – *a single transaction failure* leads to *a series of transaction rollbacks*. Considering schedule 10, none of the transactions has yet committed (so the schedule is recoverable).
- ▶ For example, If T_8 fails, T_8 must be rolled back.
 - ▶ T_9 is dependent on T_8 , T_9 must also be rolled back.
 - ▶ T_{10} is dependent on T_9 , T_{10} must be rolled back.

T_8	T_9	T_{10}
read(A)		
read(B)		
write(A)	read(A)	
	write(A)	
		read(A)
abort		

schedule 10



Cascadeless Schedules

- ▶ *Cascading rollback* can lead to the *undoing of a significant amount of work*. It is desirable to restrict the schedules to those where cascading rollbacks cannot occur.
- ▶ *Cascadeless schedules* — cascading rollbacks cannot occur. For *each pair* of transactions T_i and T_j such that T_j *reads* a data item previously *written* by T_i , the *commit* operation of T_i appears before the *read* operation of T_j .
for each (T_i, T_j) , $T_i \text{ write} \rightarrow T_j \text{ read} \Rightarrow T_i \text{ commit} \rightarrow T_j \text{ read}$
- ▶ *Every cascadeless schedule is also recoverable.*

$T_i \text{ write} \rightarrow T_j \text{ read} \Rightarrow T_i \text{ commit} \rightarrow T_j \text{ commit}$
recoverable

Transaction Isolation Levels

- ▶ ***Isolation levels*** specified by the SQL
 - ▶ ***Serializable*** — usually ensures ***serializable execution***.
 - ▶ ***Repeatable read*** — allows ***only committed data to be read*** and further requires that, between two reads of a data item by a transaction, ***no other transaction is allowed to update it***.
 - ▶ ***Read committed*** — allows ***only committed data to be read***, but does ***not require repeatable reads***. (E.g. between two reads of a data item by the transaction, another transaction may have updated the data item and committed)
 - ▶ ***Read uncommitted*** — allows ***uncommitted data to be read*** (the lowest isolation level allowed by SQL).

Transaction Isolation Levels (Cont.)

- ▶ All the isolation levels above additionally *disallow dirty writes*, that is, they *disallow writes to a data item that has already been written by another transaction that has not yet committed or aborted*.



Implementation of Isolation Levels

- ▶ We can use various *concurrency-control* policies to ensure that only acceptable schedules are generated
 - ▶ *Locking*
 - ▶ *Timestamp*
 - ▶ *Multiple versions* and *snapshot isolation*
- ▶ The *goal* of concurrency-control policies is to *provide a high degree of concurrency*, while ensuring that all schedules that can be generated are
 - ▶ *Serializable*
 - ▶ *Recoverable*
 - ▶ *Cascadeless*



Implementation of Isolation Levels (Cont.)

► *Locking*

- Instead of locking the entire database, a transaction could, instead, *lock only those data items* that it accesses.
- Under such a policy, the transaction must hold *locks long enough to ensure serializability*, but for a period *short enough not to harm performance* excessively.
- *two-phase locking protocol* (will be introduced in Chapter 15), requires a transaction to have two phases
 - one where it acquires locks but does not release any;
 - a second phase where the transaction releases locks but does not acquire any.



Implementation of Isolation Levels (Cont.)

▶ *Timestamps*

- ▶ Another category of techniques for the implementation of isolation assigns each transaction a *timestamp*, typically when it begins.
- ▶ For each data item, the system keeps *two timestamps*.
 - ▶ *Read timestamp* holds the largest (that is, the most recent) timestamp of those transactions that read the data item.
 - ▶ *Write timestamp* holds the timestamp of the transaction that wrote the current value of the data item.
- ▶ Timestamps are used to ensure that transactions *access each data item in order of the transactions' timestamps* if their accesses conflict.

Implementation of Isolation Levels (Cont.)

► *Multiple Versions & Snapshot Isolation*

- Maintaining *more than one version* of a data item.
- Allow a transaction to *read an old version of a data item* rather than a newer version written by an uncommitted transaction or by a transaction that should come later in the serialization order.
- There are a variety of multiversion concurrency-control techniques. *Snapshot isolation* is widely used in practice.
- In snapshot isolation, we can imagine that *each transaction is given its own version, or snapshot*, of the database when it begins.



Implementation of Isolation Levels (Cont.)

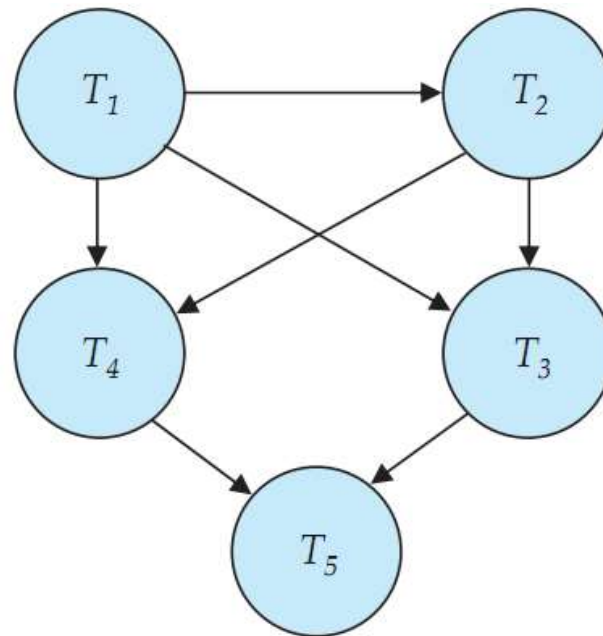
► *Multiple Versions & Snapshot Isolation*

- It *reads data from this private version* and is thus isolated from the updates made by other transactions.
- If the transaction updates the database, *update appears only in its own version*, not in the actual database itself.
- Information about these updates is saved so that the updates can be applied to the “real” database if the transaction commits.



Homework

- 14.6 Consider the precedence graph of Figure 14.16. Is the corresponding schedule conflict serializable? Explain your answer.



Homework

14.15 Consider the following two transactions:

```
T13: read(A);  
      read(B);  
      if A = 0 then B := B + 1;  
      write(B).  
T14: read(B);  
      read(A);  
      if B = 0 then A := A + 1;  
      write(A).
```

Let the consistency requirement be $A = 0 \vee B = 0$, with $A = B = 0$ the initial values.

- Show that every serial execution involving these two transactions preserves the consistency of the database.
- Show a concurrent execution of T_{13} and T_{14} that produces a nonserializable schedule.
- Is there a concurrent execution of T_{13} and T_{14} that produces a serializable schedule?